
JENSON
BIKES

ABOUT COMPANY
JENSON USA

Jenson USA is a leading American bicycle retailer founded in 1994, driven by a passion for cycling and a commitment to serving riders of all levels. We offer a wide selection of bikes, parts, gear, and accessories from top brands, backed by expert support from a team of fellow enthusiasts.

With both a strong online presence and physical locations, we deliver fast, reliable service and personalized guidance to help our customers ride with confidence.

At Jenson USA, we're not just about selling bikes-we're about building a community that rides, explores, and grows together.



BIKESTORES DATABASE

Schema Overview

stores

store_id, store_name,
phone, email, city, state

products

product_id, product_name,
brand_id, category_id, list_price

orders

order_id, customer_id,
order_date, store_id, staff_id

order_items

order_id, item_id,
product_id, quantity, discount

customers

customer_id, first_name,
last_name, email, city

staffs

staff_id, first_name,
last_name, store_id, manager_id

categories

category_id,
category_name

brands

brand_id,
brand_name

Q1 Total Products Sold by Each Store

```sql
-- 1. Find the total number of products sold by each store along with the store name

SELECT
    s.store_name,
    SUM(oi.quantity) AS total_products_sold
FROM stores s
JOIN orders o ON s.store_id = o.store_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY s.store_id, s.store_name
ORDER BY total_products_sold DESC;
```

store_name	total_products_sold
Baldwin Bikes	4779
Santa Cruz Bikes	1516
Rowlett Bikes	783

Q2 Cumulative Sum of Quantities Sold per Product

```sql
-- 2. Calculate the cumulative sum of quantities sold for each product over time

SELECT
    p.product_name,
    o.order_date,
    oi.quantity,
    SUM(oi.quantity) OVER (
        PARTITION BY p.product_id
        ORDER BY o.order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_quantity
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
ORDER BY p.product_name, o.order_date;
```

product_name	order_date	quantity	cumulative_quantity
Electra Amsterdam Fashion 3i Ladies' - 2017/2018	2018-01-01	1	1
Electra Amsterdam Fashion 3i Ladies' - 2017/2018	2018-01-21	2	3
Electra Amsterdam Fashion 3i Ladies' - 2017/2018	2018-04-30	2	5
Electra Amsterdam Fashion 7i Ladies' - 2017	2017-01-29	2	2
Electra Amsterdam Fashion 7i Ladies' - 2017	2017-02-28	1	3

Q3 Highest Total Sales Product per Category

```sql
    SELECT
        c.category_name,
        p.product_name,
        SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS total_sales,
        RANK() OVER (
            PARTITION BY c.category_id
            ORDER BY SUM(oi.quantity * oi.list_price * (1 - oi.discount)) DESC
        ) AS sales_rank
    FROM categories c
    JOIN products p ON c.category_id = p.category_id
    JOIN order_items oi ON p.product_id = oi.product_id
    GROUP BY c.category_id, c.category_name, p.product_id, p.product_name
)
SELECT
    category_name,
    product_name,
    total_sales
FROM product_sales
WHERE sales_rank = 1
ORDER BY total_sales DESC;
```

Children Bicycles	Trek Precaliber 24 21-speed Girls - 2018	0.0000
Children Bicycles	Electra Tiger Shark 3i (20-inch) - Boys' - 2018	0.0000
Comfort Bicycles	Electra Townie Balloon 3i EQ - 2017/2018	0.0000
Comfort Bicycles	Electra Townie Commute 8D Ladies' - 2018	0.0000
Cruisers Bicycles	Electra Cruiser 1 Tall - 2016/2018	0.0000

Q4 Customer Who Spent the Most Money

```sql
-- 4. Find the customer who spent the most money on orders

SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_spent DESC
LIMIT 1;
```

customer_id	first_name	last_name	total_spent
100	Kellie	Franco	0.0000

Q5 Highest-Priced Product per Category

```sql
5. Find the highest-priced product for each category name

WITH ranked_products AS (
    SELECT
        c.category_name,
        p.product_name,
        p.list_price,
        RANK() OVER (
            PARTITION BY c.category_id
            ORDER BY p.list_price DESC
        ) AS price_rank
    FROM categories c
    JOIN products p ON c.category_id = p.category_id
)
SELECT
    category_name,
    product_name,
    list_price
FROM ranked_products
WHERE price_rank = 1
ORDER BY list_price DESC;
```

category_name	product_name	list_price
Road Bikes	Trek Domane SLR 9 Disc - 2018	1199999.00
Mountain Bikes	Trek Fuel EX 98 275 Plus - 2017	529999.00
Mountain Bikes	Trek Remedy 98 - 2017	529999.00
Electric Bikes	Trek Powerfly 8 FS Plus - 2017	499999.00
Electric Bikes	Trek Powerfly 7 FS - 2018	499999.00
Electric Bikes	Trek Super Commuter + 8S - 2018	499999.00
Cyclocross Bicycles	Trek Boone 7 Disc - 2018	399999.00
Cruisers Bicycles	Electra Townie Commute Go! - 2018	299999.00

Q6 Total Orders per Customer per Store

```sql
-- 6. Find the total number of orders placed by each customer per store

SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    s.store_name,
    COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN stores s ON o.store_id = s.store_id
GROUP BY c.customer_id, c.first_name, c.last_name, s.store_id, s.store_name
ORDER BY c.customer_id, total_orders DESC;
```

customer_id	first_name	last_name	store_name	total_orders
1	Debra	Burks	Baldwin Bikes	3
2	Kasha	Todd	Santa Cruz Bikes	3
3	Tameka	Fisher	Santa Cruz Bikes	3
4	Daryl	Spence	Baldwin Bikes	3
5	Charolette	Rice	Santa Cruz Bikes	3
6	Lyndsey	Bean	Baldwin Bikes	3
7	Latasha	Hays	Baldwin Bikes	3
8	Jacquline	Duncan	Baldwin Bikes	3

Q7 Staff Members with No Sales

```sql
-- 7. Find the names of staff members who have not made any sales

SELECT
    st.staff_id,
    st.first_name,
    st.last_name,
    st.email
FROM staffs st
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.staff_id = st.staff_id
)
ORDER BY st.staff_id;
```

staff_id	first_name	last_name	email
1	Fabiola	Jackson	fabiolajackson@bikesshop
4	Virgie	Wiggins	virgiewiggins@bikesshop
5	Jannette	David	jannettedavid@bikesshop
10	Bernardine	Houston	bernardinehouston@bikesshop

Q8 Top 3 Most Sold Products by Quantity

```sql
-- 8. Find the top 3 most sold products in terms of quantity

SELECT
    p.product_id,
    p.product_name,
    SUM(oi.quantity) AS total_quantity_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_quantity_sold DESC
LIMIT 3;
```

product_id	product_name	total_quantity_sold
6	Surly Ice Cream Truck Frameset - 2016	167
13	Electra Cruiser 1 (24-Inch) - 2016	157
16	Electra Townie Original 7D EQ - 2016	156

Q9 Median Value of the Price List

```sql
-- 9. Find the median value of the price list

WITH ordered_prices AS (
    SELECT
        list_price,
        ROW_NUMBER() OVER (ORDER BY list_price) AS row_num,
        COUNT(*) OVER () AS total_count
    FROM products
)
SELECT
    AVG(list_price) AS median_price
FROM ordered_prices
WHERE row_num IN (FLOOR((total_count + 1) / 2.0), CEIL((total_count + 1) / 2.0));
```

median_price
74999.000000

Q10 Products Never Ordered (using EXISTS)

```sql
-- 10. List all products that have never been ordered (using EXISTS)

SELECT
    p.product_id,
    p.product_name,
    p.list_price,
    c.category_name,
    b.brand_name
FROM products p
JOIN categories c ON p.category_id = c.category_id
JOIN brands b ON p.brand_id = b.brand_id
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
)
ORDER BY p.product_name;
```

product_id	product_name	list_price	category_name	brand_name
284	Electra Savannah 1 (20-inch) - Girl's - 2018	31999.00	Children Bicycles	Electra
291	Electra Sweet Ride 1 (20-inch) - Girl's - 2018	31999.00	Children Bicycles	Electra
195	Electra Townie Go! 8i Ladies' - 2018	259999.00	Electric Bikes	Electra
121	Surly Krampus Frameset - 2018	249999.00	Mountain Bikes	Surly
1	Trek 820 - 2016	37999.00	Mountain Bikes	Trek
316	Trek Checkpoint ALR 4 Women's - 2019	169999.00	Road Bikes	Trek

Inventory Management Analysis
Key Insight

 Derived Insight	 Actionable Suggestion
1. Some products have never been ordered	Discontinue or rebrand underperforming SKUs; analyze if pricing, visibility, or relevance is an issue.
2. A few products dominate sales in each category (from top-selling product query)	Increase stock for top-performing items; consider highlighting them in marketing efforts.
3. Cumulative sales data shows seasonality or surges (from cumulative quantity query)	Use this time-based trend to improve demand forecasting and optimize restocking schedules.

Q11 Staff with Above-Average Sales Count

```sql
-- 11. List the names of staff members who have made more sales than the average number of sales by all staff members
WITH staff_sales AS (
    SELECT
        st.staff_id,
        st.first_name,
        st.last_name,
        COUNT(o.order_id) AS sales_count
    FROM staffs st
    LEFT JOIN orders o ON st.staff_id = o.staff_id
    GROUP BY st.staff_id, st.first_name, st.last_name
),
avg_sales AS (
    SELECT AVG(sales_count) AS average_sales
    FROM staff_sales
)
SELECT
    ss.staff_id,
    ss.first_name,
    ss.last_name,
    ss.sales_count
FROM staff_sales ss
CROSS JOIN avg_sales a
WHERE ss.sales_count > a.average_sales
ORDER BY ss.sales_count DESC;
```

staff_id	first_name	last_name	sales_count
6	Marcelene	Boyer	553
7	Venita	Daniel	540
3	Genna	Serrano	184

Staff Performance Analysis
Key Insight

Derived Insight	Actionable Suggestion
1. Some staff members have recorded no sales	Investigate possible reasons: role mismatch, inadequate training, or lack of assignments; consider mentoring or reassignment.
2. Sales distribution among staff is uneven	Set realistic sales targets and balance workloads to ensure fair opportunities for all team members.
3. Few staff exceed average sales significantly	Identify and reward high performers; use their strategies as training material for others.

Q12 Customers Who Ordered from Every Category

```sql
SELECT c.first_name + ' ' + c.last_name
        AS customer_name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id
GROUP BY c.first_name, c.last_name
HAVING COUNT(DISTINCT p.category_id) =
  (SELECT COUNT(*) FROM categories);
```

	customer_name
▶	0